ACME — Research & Recommendation

Goal: advise whether NHI should build an ACME-compatible service, integrate with existing CAs (e.g., Let's Encrypt), or partner with enterprise CA providers — and propose a safe, pragmatic approach.

Technical overview

What is ACME?

ACME (Automatic Certificate Management Environment) is a standard protocol (RFC) that automates certificate issuance and renewal between an *ACME client* (the requester) and an *ACME server/CA*. Common use-cases: automated issuance, automatic renewal before expiry, and challenge-based domain validation (HTTP-01, DNS-01, TLS-ALPN-01).

How certificate automation works (high level)

1. **Client requests certificate**: ACME client creates a key pair and requests a certificate for one or more identifiers (domain names).
2. **CA issues challenges**: CA issues validation challenges (HTTP-01, DNS-01, TLS-ALPN-01) to prove control over the domain.
3. **Client proves control**: Client responds to challenge (e.g., place token on HTTP path or create DNS TXT record).
4. **CA validates and issues certificate**: Once validated, CA signs and returns the certificate chain.
5. **Renewal**: Client automatically repeats the flow before expiry to rotate certs. Good implementations stagger renewals and keep overlapping validity to avoid outages.

Key technical components for NHI

- **ACME client component** (orchestrator): coordinates requests, challenge handling, rotation schedules, retries, and audit logs.
- **Challenge handlers**:
  - **HTTP-01**: requires routing traffic to an HTTP challenge endpoint (works for public web services).
  - **DNS-01**: requires DNS provider API access (works for wildcard certificates and internal domains when DNS control is available).
  - **TLS-ALPN-01**: less common, requires serving specific cert on TLS port for validation.
- **Secrets management / key storage**: private keys must be stored securely (HSM/KMS/secret store).
- **Policy & orchestration layer**: enforces TTL, approval gates, allowed domains, multi-tenant isolation, and CA selection.
- **Observability & logging**: monitor renewal success, expiry windows, CA rate-limits and failures, and full audit trail.
- **Fallback / failover**: ability to fall back to alternative CA or alerting if issuance fails.

Build vs Integrate vs Partner — comparative analysis

We evaluate Complexity, Security Risks, Time-to-Market, Cost, and Competitive Differentiation.

Option A — Build ACME-compatible service in-house

What: Implement an ACME server or full ACME CA stack (i.e., become a CA) and/or a complete ACME orchestration + issuance pipeline under your control.

Complexity

- Very high. Running a CA is complex (policy, auditing, root/intermediate key management) and building a hardened ACME server + CA stack requires deep PKI expertise.
- Even if only building an *orchestrator* (client-side + proxy), still non-trivial: challenge handling, multi-DNS integration, robust retry/queueing, tenant isolation.

Security risks

- High liability: if you act as a CA, key compromise or mis-issuance has severe consequences.
- Increased compliance obligations (audits, CRL/OCSP infrastructure, incident response).
- Secrets management + HSM requirements.

Time-to-market

- Long. Maturity requires months → years (especially if becoming a trusted CA).

Cost implications

- High initial engineering investment, ongoing ops and compliance cost, HSMs, audits, liability insurance.

Competitive differentiation

- High potential control: you can craft custom policies, internal PKI features, deep integration with platform workflows.
- But differentiation may not justify cost/risk unless PKI is core to product strategy.

When build is sensible

- If NHI requires full control over a private/internal PKI for regulatory reasons, or if certificate issuance is central to your product value proposition and will be used as a paid feature for enterprise customers.

Option B — Integrate external ACME CAs

What: Use existing public ACME CAs for automated issuance; implement an orchestration layer that automates ACME client flows and handles challenge automation for customers.

Complexity

- Moderate. ACME protocol is well-documented and mature; common ACME clients/libraries exist.
- Need to implement challenge automation (especially DNS-01) across DNS providers and multi-tenant orchestration.

Security risks

- Lower than building a CA. Still need secure key storage for issued cert keys, and strict multi-tenant isolation.
- Dependence on CA limitations (rate limits, issuance policies) and CA availability.

Time-to-market

- Fast. You can go live quickly for public-facing domains (HTTP-01) and with DNS provider integration for wildcard coverage.

Cost implications

- Low to moderate: Let's Encrypt is free, but operational costs include engineering to automate DNS challenges, KMS/HSM usage, and monitoring.
- No CA licensing fees, but watch for DNS provider API fees or third-party DNS automation costs.

Competitive differentiation

- Moderate. Fast delivery is beneficial; perhaps not a strong differentiator on its own, but mature automation and UX can be a product advantage.

When integrate is sensible

- For automating public-facing certificates at scale and providing immediate value to customers with minimal regulatory burden.

Key caveats

- Let's Encrypt rate limits require architectural care (caching, reuse, wildcard issuance via DNS-01).
- Some enterprise customers require private/internal PKI (not covered by public CAs).

Option C — Partner with enterprise CA providers

What: Integrate with established enterprise PKI vendors (Venafi, DigiCert, Entrust, Microsoft CA, HashiCorp + private PKI, etc.) either via APIs or managed service partnerships.

Complexity

- Moderate to high depending on provider APIs and enterprise feature set. Integration work includes provisioning, approvals, enrolment, and syncing audit data.
- Handling multiple partner integrations increases integration complexity.

Security risks

- Lower CA-side risk since partner handles CA operations; still need secure integration and multi-tenant isolation.
- Third-party SLAs and operational considerations.

Time-to-market

- Medium. Integrations can take effort but often come with dedicated support and enterprise features out-of-the-box.

Cost implications

- Higher recurring costs (licensing, per-certificate fees, or partnership fees). Potential revenue-sharing or reseller arrangements possible.
- Lower compliance overhead for the platform if partner assumes CA responsibilities.

Competitive differentiation

- High for enterprises: provides compliance, extended features (policy engines, HSM-backed CA, enterprise support), and stronger SLAs.
- Can be positioned as premium offering for enterprise customers.

When partner is sensible

- When targeting enterprise customers who demand auditability, compliance, private PKI, and vendor-backed SLAs.

Recommendation

Recommended approach — Hybrid (Orchestrate + Integrate + Partner):

- Primary approach: Build a central ACME orchestration layer (client/orchestrator) inside NHI that automates issuance/renewal workflows, challenge handling, scheduling, retrying, auditing, and secrets management.
- Integrations: Connect the orchestrator to public ACME CAs (e.g., Let’s Encrypt) for public/external certificates and to enterprise CA providers (via API/partnerships) for customers requiring internal PKI, compliance, and higher SLAs.

- Do not become a public CA. Avoid building a full CA unless there is a clear, strategic necessity and the organization can accept the long-term operational and compliance burden.

Rationale

- Fastest path to delivering customer value (use mature CAs for issuance).
- Retains central control (policy, audit, multi-tenant orchestration) and consistent UX across CA backends.
- Supports a broad market: public websites (cheap/fast) + enterprise customers (paid, compliant).
- Avoids CA-level liabilities while enabling differentiation through orchestration, policy controls, RBAC, and observability.

Risks & Mitigations

Risk: CA rate limits and outages (Let's Encrypt limits)

Mitigation

- Implement certificate reuse, wildcard certs (via DNS-01) where possible.
- Stagger renewals and queue requests to avoid bursts.
- Cache and reuse challenge tokens and certificates when allowed.
- Failover to alternative CA/provider when supported by policy.

Risk: Private key compromise or leakage

Mitigation

- Use HSMs or cloud KMS for private key storage.
- Rotate keys on compromise; short-lived certs where possible.
- Strict IAM & audit controls around key access.

Risk: DNS provider/API complexity and security

Mitigation

- Build a pluggable DNS provider adapter layer with robust credential management.
- Prefer provider-managed credentials stored in KMS; minimize broad-scope API keys.
- Validate DNS changes and implement least-privilege access.

Risk: Multi-tenant isolation & accidental cross-tenant issuance

Mitigation

- Strong tenant scoping in orchestration layer using tenant IDs, per-tenant credentials.

- End-to-end tests and policy-as-code checks.
- Audit logging for all issuance and challenge actions.

Risk: Compliance & enterprise requirements (auditability, CRL/OCSP)

Mitigation

- For enterprise customers, integrate with partner CAs that provide required compliance artifacts and SLAs.
- Store and surface revocation/OCSP status in inventory; support pinned policies.

Phased rollout strategy (no time estimates — phases only)

Phase 0 — Discovery & architecture

- Requirements & stakeholder interviews (public domains vs internal).
- Identify DNS providers to support and enterprise customers' PKI needs.
- Design orchestration, secrets management, audit model, and multi-CA abstraction.

Phase 1 — Minimal viable automation (public web)

- Implement ACME orchestrator + HTTP-01 automation for public sites using Let's Encrypt.
- Secure key storage (KMS/HSM) and basic audit logs.
- UI/alerts for rotation failures and expiry warnings.
- Beta with a small set of tenants.

Phase 2 — DNS-01 / wildcard + scale hardening

- Add DNS-01 integrations for wildcard certificates (DNS provider adapters).
- Implement rate-limit handling, queuing, backoff, and renewal staggering.
- Improve observability and SLOs.

Phase 3 — Enterprise CA integrations & partner offerings

- Integrate 1–2 enterprise CA providers for paid/compliance customers (API enrollment, approval workflows).
- Add tenant-level CA selection policy and billing/entitlement hooks.
- Add richer audit and compliance reports.

Phase 4 — Advanced features & resilience

- Multi-CA failover, certificate canaries, private/internal PKI support if required.
- Policy engine (approval gates, domain allowances), RBAC expansion, approval workflows.
- Full runbook, incident response playbooks, and compliance audits.

Phase 5 — Ops maturity

- SLA formalization, capacity planning, security audits, external compliance certifications (if required).
- Expand partner network and potential resale models.

Operational & success criteria (what to measure)

- **Renewal success rate**: % certs renewed automatically before expiry.
- **Expiry incidents**: number of production outages caused by expired certs. Target: zero.
- **Time-to-issue**: median time from request to issued cert (measure by CA and challenge type).
- **Rate-limit incidents**: number of blocked issuance attempts due to CA limits.
- **Audit completeness**: % actions with full audit trail available.
- **Customer satisfaction**: NPS / CSAT for enterprise customers using automated rotation.

Final verdict

- **Do not build a public CA.** The CA role is high-risk, costly, and requires specialized ops.
- **Do build an orchestration layer (in-house)** that centralizes policy, secrets, audit logs, scheduling, and multi-CA adapters. This gives control and product differentiation with lower risk.
- **Integrate with Let's Encrypt / ACME CAs** for public domains for fast time-to-market and low cost.
- **Partner with enterprise CA providers** for customers needing compliance, private PKI, SLAs, or advanced features; position that as a premium offering.

This hybrid model balances speed, cost, security, and market coverage while enabling NHI to differentiate via orchestration, policy, RBAC, and operational excellence.